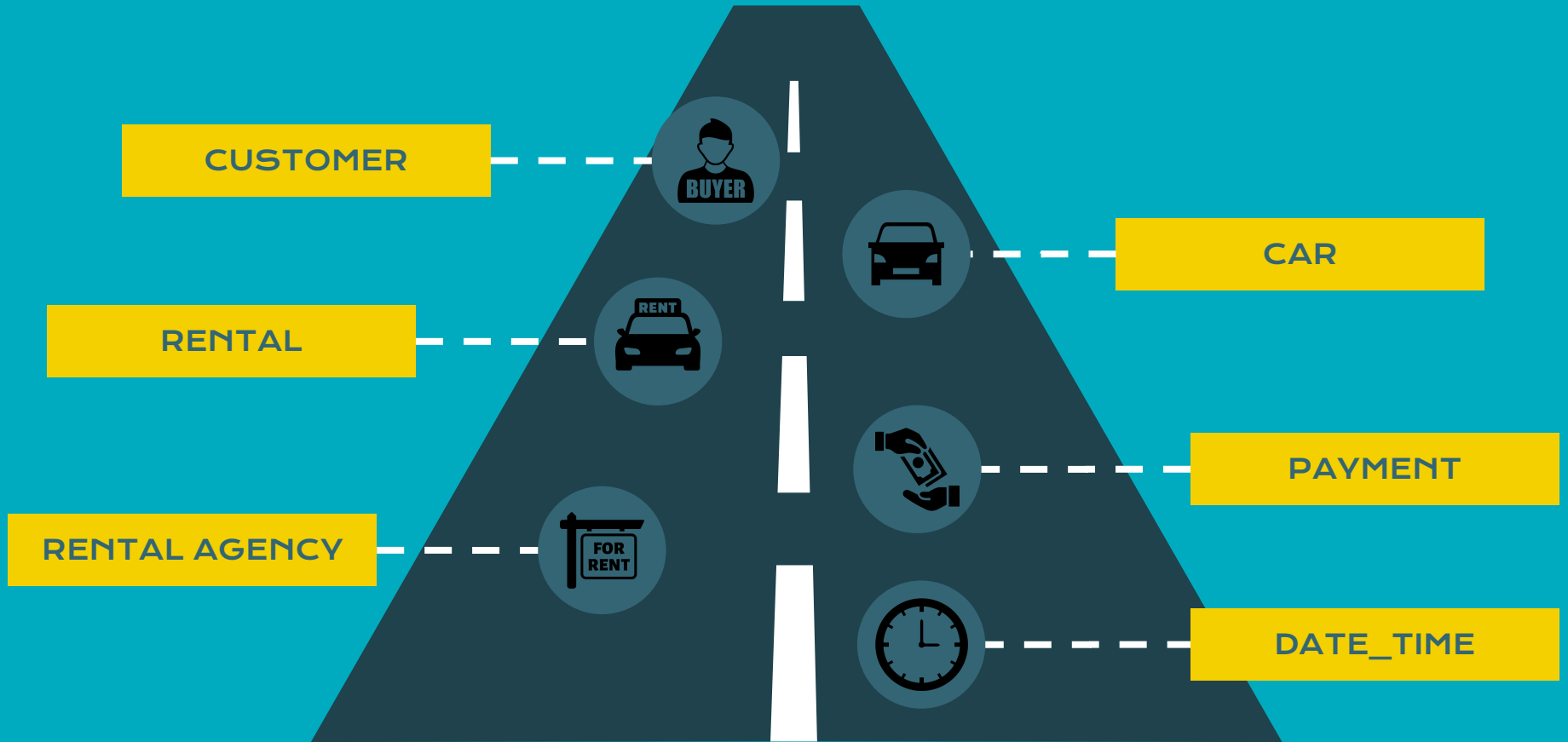
Car Rental System

Made by:
Yousuf Al-Busaidi(24-0268)
Hussain Al-Lawati(24-0359)
Said Al-Mukhaini(24-0137)
Anas Al Azri(24-0328)

Description

The Car Rental System is an application whose purpose is to easily help customers rent vehicles through the use of an automated system. Customers can select from three types of vehicles which are Sedan, SUV and Truck and book them with a specific start date and return date and make payments through the system. The system automatically calculates the total rental cost based on the number of days and the type of car rented and applies a late return fee if the car is returned after the expected date. At the same time the agency can track all the vehicles currently rented, view all registered customers and manage all rental transactions and payments. The Car Rental System takes care of the whole process of renting a car starting from registering as customer to selecting a vehicle to processing a payment to returning the car. This system is more efficient compared to manual car rentals as it eliminates human error and automates the entire rental process.

Classes


CUSTOMER
BUYER
CAR
RENTAL
RENT
PAYMENT
RENTAL AGENCY
FOR RENT
DATE_TIME

Each class and its responsibility

- Customer: The customer class represents a person who wants to rent a car from us. It stores all the customers personal information needed to identify the customer and process his rental. Every rental transaction is linked back to the customer object.
- Car: The car class represents a vehicle available for rent in the agencies collection. It keeps track of the cars details and whether its available or already rented out. The cost of rental varies depending on the type of car.
- Rental: The rental class is the core transaction of the system. It connects a customer to a car and records the start date and end date and automatically calculates the total based on the duration the car was rented. It also tracks whether the rental is still active or closed.
- Payment: The payment class takes care of the financial aspect of the system. When a customer confirms the rental of a certain vehicle they complete the payment by using their preferred payment method. The transaction is processed in the payment class and a receipt will be generated for the customer.
- Rental Agency: This class controls the entire system. All the main functions of the system from cars to customers to payments happens in this class.
- Date_Time: This class tracks the exact date and time of all rentals in the system and all the transactions are stored and managed in the system. It records when the car is rented and returned and calculates the duration of the rental to find the total cost, if the car is returned after the expected date, the class adds an additional fee.

Class Relationships

Our system will use two types of relationships between classes: Inheritance and composition

We will have two inheritance relationships:

1. Person → Customer: The Customer class will inherits from Person. This means Customer will automatically get the attributes of person like name, ID and phone number from person and then it will have its own attributes like drivers license number and others.
2. Car → Sedan, SUV, Truck: We will have sub classes called Sedan, SUV and Truck and they will all inherit from Car. They share the same basic car attributes but each one calculates its price differently depending on the type of car they rent.

   Car → Abstract Class: The Car class is an abstract class because it contains a pure virtual function called calculatePrice(). This means you cannot create a Car object directly. Instead you must create either a Sedan, SUV or Truck. This forces every car type to have its own version of calculateprice() which calculates the rental cost differently depending on the car type.

Class Relationships

Our system will have 6 composition relationships:

1. RentalAgency has Cars: The agency will own and manage all the cars in our collection using a vector Car pointers
2. RentalAgency has Customers: The agency will store and manage all the registered customers using vector of Customer pointers
3. RentalAgency has Rentals: The agency creates and tracks all the rental transactions using a vector of Rental pointers
4. RentalAgency has Payments: The agency stores all the payment records for every completed rental using a vector of Payment pointers
5. Rental has a Car: Every rental will be linked to one specific car being rented using car pointer
6. Rental has a Customer: Every rental is linked to a customers who made the reservation using a Customer pointer
7. Rental has a Date_Time: Every rental stores a start date and time and an expected return date and time using the Date_Time class to track the rental period and calculate the total cost

System Features

Our Car Rental System includes the following features:

- Add cars to the system(Sedan, SUV, Truck) with a unique Car ID and plate number
- Register customers with their personal details and driving license
- Rent a car by linking a customer to a car with a start date and expected return date
- Automatically calculate the total rental cost based on number of days and car type
- Return a car and detect late returns with an automatic late fee applied
- Process payments using Cash, Credit Card or Debit Card and generate an invoice
- View all cars, customers and rentals in the system at any time

Code structure

Our system is built using 10 classes:

- Person and Customer handle all customer information
- Car, Sedan, SUV and Truck handle all vehicle information
- Date_Time manages all date and time operations
- Rental connects a car to a customer and tracks the transaction
- Payment handles all financial transactions
- RentalAgency controls the entire system through a menu

Use of OOP concepts

Concept	How we used it
Encapsulation	All attributes are private with public getter and setters
Inheritance	Customers inherits from Person,Sedan/SUV/Truck inherit from car
Abstraction	Car is an abstract class with pure virtual calculatePrice()
Polymorphism	Sedan,SUV and Truck each override calculatePrice() and displayCar()
Composition	Rental has Car, Customer and Date_Time
Pointers	Car* and Customer* used in Rental and RentalAgency
Vectors	RentalAgency stores all objects in vectors
Static	carCount and rentalCount auto generate unique IDs
Exception Handling	try/catch blocks in RentalAgency catch all errors
Operator Overloading	operator== in Rental compares two rentals by ID
Destructors	~RentalAgency() and ~Rental() clean up all pointers